

Relatório de Melhorias Implementadas

Sistema Mesa de Estudos: evolução do ciclo de estudos para uma experiência mais inteligente, com assistente, agenda, questões, revisão adaptativa e recomendações automáticas.

DATA

03/05/2026

PROJETO

Mesa de Estudos

AMBIENTE

Next.js + Prisma + PostgreSQL

1. Resumo Executivo

As melhorias feitas hoje criam uma camada de inteligência sobre o sistema existente. Antes, o sistema já organizava ciclos, revisões e desempenho. Agora, ele passa a interpretar sinais do usuário e sugerir a próxima melhor ação, aproximando a experiência de um assistente de estudos.

Objetivo alcançado: reduzir o trabalho manual do usuário e deixar o sistema mais proativo na organização do estudo.

2. Melhorias Implementadas

2.1 Ciclo adaptativo e rebalanceamento

Arquivos principais: `lib/cicloAlgorithm.ts`, `service/ciclo.service.ts`, `src/app/api/ciclos/route.ts`.

- O algoritmo passou a aceitar um fator de `desempenho` por disciplina.
- Disciplinas com baixo avanço, poucos registros ou muitos dias sem estudo ganham prioridade.
- Disciplinas quase concluídas recebem menor prioridade.
- Foi criado o botão **Rebalancear** na tela de Ciclos de estudo.
- O rebalanceamento é manual e conservador, evitando mudar a fila de forma agressiva.

Valor para o usuário: o ciclo deixa de ser apenas estático e passa a reagir ao progresso real do estudante.

2.2 Minha Mesa com ações imediatas

Arquivo principal: `src/app/minha-mesa/page.tsx`.

- Adição da meta do dia.
- Exibição de sessões concluídas hoje.
- Exibição de revisões pendentes e atrasadas.
- Novas ações: **Pular** e **Remarcar**.
- Concluir sessão continua marcando tópico como concluído.
- Pular/remarcar avançam a fila sem concluir tópico.

Valor para o usuário: a tela vira um centro de execução diária, não apenas um cronômetro.

2.3 Desempenho como central de inteligência

Arquivos principais: `src/app/api/desempenho/route.ts`, `src/app/desempenho/page.tsx`.

- Criação de tendência semanal: ritmo subindo, estável ou caindo.
- Identificação de pontos fracos por disciplina.

- Recomendações automáticas com base nos sinais do usuário.
- Bloco de desempenho por tópico.
- Integração com dados de questões quando disponíveis.

Valor para o usuário: o painel passa a explicar onde focar, em vez de apenas mostrar números.

2.4 Assistente de Estudo

Arquivos principais: `service/assistente-estudo.service.ts`, `src/app/api/assistente/route.ts`.

- Criação de um serviço central para decidir a próxima melhor ação.
- O assistente avalia revisões, ciclo, prioridades, desempenho e questões.
- Ele pode recomendar revisão, ciclo, reforço, questões ou criação de ciclo.
- A orientação aparece na Minha Mesa e na Agenda.

Valor para o usuário: o sistema começa a responder “o que eu devo fazer agora?”.

2.5 Motor de prioridades

Arquivo principal: `service/prioridade.service.ts`.

- Calcula score de prioridade por disciplina.
- Considera percentual concluído, tempo registrado, sessões feitas e dias sem estudo.
- Gera motivos explicáveis, como “baixo avanço” ou “7 dias sem estudo”.

Valor para o usuário: as recomendações deixam de parecer aleatórias e passam a ter justificativa.

2.6 Revisão adaptativa

Arquivos principais: `service/revisao-adaptativa.service.ts`, `src/app/api/revisoes/route.ts`.

- Revisões agora aceitam resultado: `facil`, `medio`, `dificil` ou `errei`.
- O próximo intervalo muda conforme o resultado.
- Foram criados status específicos: `REVISAO_FACIL`, `REVISAO_MEDIO`, `REVISAO_DIFICIL` e `REVISAO_ERREI`.

Valor para o usuário: o sistema revisa mais cedo o que foi difícil e espaça melhor o que foi fácil.

2.7 Módulo Questões

Arquivos principais: `service/questoes.service.ts`, `src/app/api/questoes/route.ts`, `src/app/questoes/page.tsx`.

- Novo menu **Questões**.
- Nova página para registrar baterias de questões.
- Registro de total de questões, acertos, erros e percentual.
- Diagnóstico de disciplina crítica.
- Histórico recente de baterias.
- Integração com Desempenho, Agenda e Assistente de Estudo.

Valor para o usuário: o sistema passa a saber se o aluno realmente aprendeu, não apenas se estudou.

2.8 Agenda inteligente

Arquivos principais: `service/agenda.service.ts`, `src/app/api/agenda/route.ts`, `src/app/agenda/page.tsx`.

- Novo menu **Agenda**.
- Plano do dia.
- Sessões previstas.
- Revisões vencidas e futuras.
- Questões recomendadas.

- Histórico por dia.
- Identificação de dias sem estudo.
- Meta semanal.
- Previsão de conclusão do edital.
- Destaque de prova/data do edital.
- Opção de remarcar a sessão atual.

Valor para o usuário: a agenda responde “quando faço?” e organiza a semana de forma prática.

3. Novas APIs Criadas ou Alteradas

Endpoint	Função	Observação
GET /api/assistente	Retorna a próxima melhor ação do usuário.	Usado por Minha Mesa e Agenda.
GET /api/agenda	Consolida plano do dia, semana, revisões, questões e edital.	Usado pela nova página Agenda.
GET /api/questoes	Retorna resumo, tópicos críticos e histórico de questões.	Funciona quando a tabela existe.
POST /api/questoes	Registra uma bateria de questões.	Requer tabela <code>questao_treino</code> .
PATCH /api/ciclos	Agora aceita ações: concluir, pular, remarcar e rebalancear.	Sem corpo mantém o comportamento de concluir sessão.
POST /api/revisoes	Aceita resultado da revisão.	Permite revisão adaptativa.

4. O Que Precisa Existir no Banco Real

4.1 Tabela obrigatória para o módulo Questões

Para que o menu **Questões**, a Agenda, o Desempenho e o Assistente usem acertos/erros sem erro, é necessário criar a tabela abaixo no schema `planejamento`.

O código não tenta mais criar essa tabela automaticamente, porque o usuário atual do banco não tem permissão de `CREATE` no schema `planejamento`. O SQL deve ser executado por um usuário com permissão.

```
CREATE TABLE IF NOT EXISTS planejamento.questao_treino (  
  id_questao_treino BIGSERIAL PRIMARY KEY,  
  id_usuario BIGINT NOT NULL,  
  id_disciplina INT NOT NULL,  
  id_topico BIGINT NULL,  
  total_questoes INT NOT NULL,  
  total_acertos INT NOT NULL,  
  percentual NUMERIC(5,2) NOT NULL,  
  data_registro TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT questao_treino_total_check CHECK (total_questoes > 0),  
  CONSTRAINT questao_treino_acertos_check CHECK (total_acertos >= 0 AND total_acertos <= total_questoes)  
);  
  
CREATE INDEX IF NOT EXISTS idx_questao_treino_usuario  
  ON planejamento.questao_treino (id_usuario);  
  
CREATE INDEX IF NOT EXISTS idx_questao_treino_disciplina  
  ON planejamento.questao_treino (id_usuario, id_disciplina);  
  
CREATE INDEX IF NOT EXISTS idx_questao_treino_topico  
  ON planejamento.questao_treino (id_usuario, id_topico);
```

Arquivo criado com esse SQL: `scripts/create-questoes-table.sql`.

4.2 Coluna recomendada para alinhar Prisma e banco real

Foi identificado que o `schema.prisma` declara a coluna `planejamento.sessao_estudo.id_ciclo_disciplina`, mas ela não existe no banco real. Isso causou erro na Agenda quando o Prisma tentou ler a tabela inteira.

AAgenda já foi corrigida com SQL seletivo, mas para deixar o sistema perfeito e evitar erros futuros, a recomendação é alinhar o banco real adicionando a coluna.

```
ALTER TABLE planejamento.sessao_estudo
  ADD COLUMN IF NOT EXISTS id_ciclo_disciplina BIGINT NULL;

CREATE INDEX IF NOT EXISTS idx_sessao_estudo_ciclo_disciplina
  ON planejamento.sessao_estudo (id_ciclo_disciplina);

ALTER TABLE planejamento.sessao_estudo
  ADD CONSTRAINT fk_sessao_estudo_ciclo_disciplina
  FOREIGN KEY (id_ciclo_disciplina)
  REFERENCES planejamento.ciclo_disciplina (id_ciclo_disciplina)
  ON DELETE SET NULL;
```

Por que isso importa: essa coluna permite vincular uma sessão realizada ao slot exato do ciclo que a originou.

4.3 Status novos usados pelo sistema

Se o banco tiver constraint rígida para `sessao_estudo.status`, ela precisa aceitar os novos valores:

- `CONCLUIDA`
- `REVISAO`
- `REVISAO_FACIL`
- `REVISAO_MEDIO`
- `REVISAO_DIFICIL`
- `REVISAO_ERREI`
- `PULADA`
- `REMARCADA`

No `schema.prisma` há comentários indicando que algumas tabelas têm check constraints no banco. Por isso, é importante conferir se a constraint de status existe no banco real.

4.4 Permissões necessárias

O usuário da aplicação precisa ter permissões para:

- `SELECT`, `INSERT` em `planejamento.questao_treino`;
- `SELECT`, `INSERT`, `UPDATE` em tabelas de progresso, sessões e ciclos;
- `SELECT` em tabelas de concurso, disciplina, tópico, cargo e edital;
- `USAGE` no schema `planejamento` e `concurso`.

5. Arquivos Criados Hoje

- `service/agenda.service.ts`
- `service/assistente-estudo.service.ts`
- `service/prioridade.service.ts`
- `service/questoes.service.ts`
- `service/revisao-adaptativa.service.ts`
- `src/app/api/agenda/route.ts`
- `src/app/api/assistente/route.ts`

- `src/app/api/questoes/route.ts`
- `src/app/agenda/page.tsx`
- `src/app/questoes/page.tsx`
- `scripts/create-questoes-table.sql`

6. Arquivos Alterados Hoje

- `lib/cicloAlgorithm.ts`
- `service/ciclo.service.ts`
- `src/app/api/ciclos/route.ts`
- `src/app/api/desempenho/route.ts`
- `src/app/api/revisoes/route.ts`
- `src/app/ciclos/page.tsx`
- `src/app/configuracoes/page.tsx`
- `src/app/dashboard/page.tsx`
- `src/app/desempenho/page.tsx`
- `src/app/minha-mesa/page.tsx`
- `src/app/perfil/page.tsx`
- `src/app/revisoes/page.tsx`

7. Validações Realizadas

- `npx.cmd tsc --noEmit` : executado com sucesso.
- `npm.cmd run lint` : executado com sucesso, apenas com warnings antigos.
- Servidor local continuou ativo em `http://localhost:3000`.

8. Próximos Passos Recomendados

1. Executar `scripts/create-questoes-table.sql` no banco real.
2. Adicionar `id_ciclo_disciplina` na tabela real `planejamento.sessao_estudo`.
3. Conferir constraints de `status` em `sessao_estudo`.
4. Depois disso, testar os fluxos: Questões, Agenda, Revisões, Minha Mesa e Rebalancear ciclo.

Conclusão: com as tabelas e campos acima criados no banco real, o sistema fica preparado para operar de forma mais inteligente, com recomendações automáticas e menos trabalho manual para o usuário.